线段检测

线段检测
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码讲解
- 检测线段
  - find_line_segments
  - 霍夫变换

例程实验效果简介

本节例程代码位于：【源码汇总 / 04.Detecting / 01.find_lines】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮，

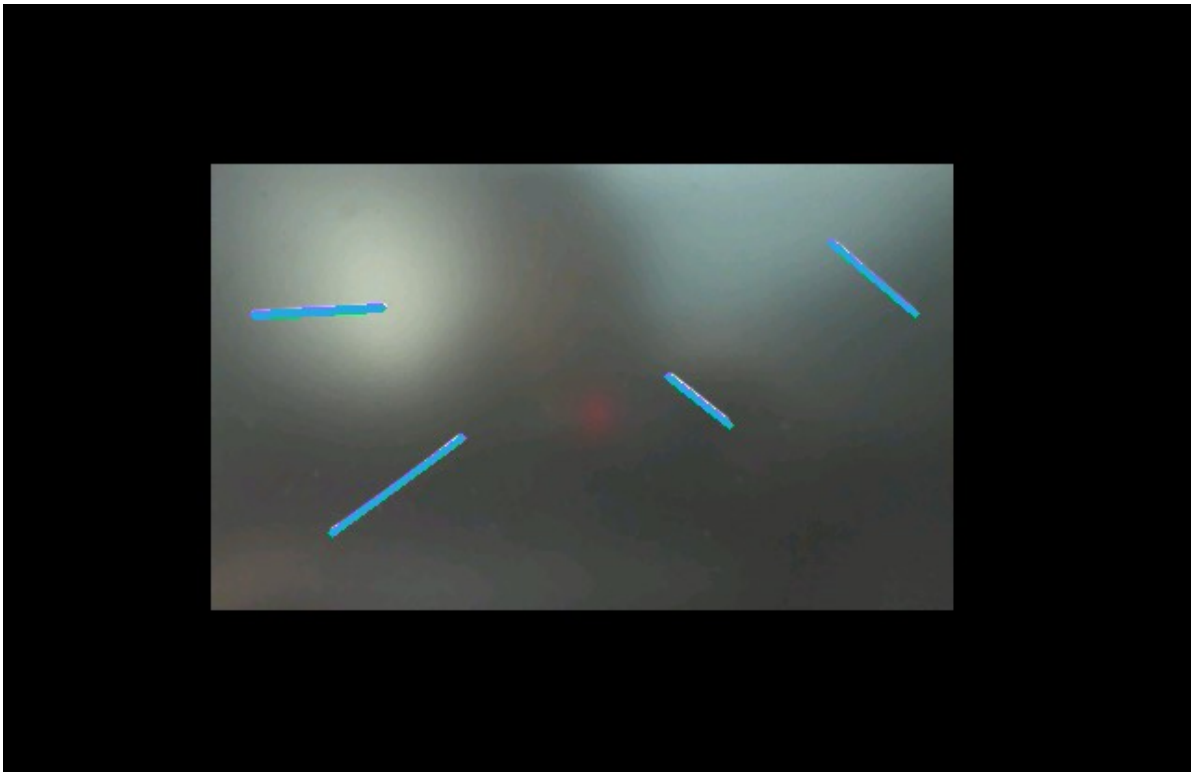
将K230的摄像头对准线段

可以看到屏幕上会标记出画面中的**线段**（如果没有屏幕就看帧缓冲区）

原图：



K230识别结果：



代码讲解

本节我们要用的的外设主要是摄像头模块

线段检测由 K230中的 find_line_segments() 方法实现，该方法属于image模块

例程使用的代码如下：

完整代码

*以【源码汇总 / 04.Detecting / 01.find_lines.py】文件内容为准

```
# 导入必要的模块：时间、操作系统、系统、垃圾回收
# (Import required modules: time, operating system, system, garbage collection)
import time, os, sys, gc
# 导入媒体相关模块：传感器、显示、媒体管理
# (Import media-related modules: sensor, display, media management)
from media.sensor import *
from media.display import *
from media.media import *
# 导入PipeLine库，用于图像处理Pipeline和性能计时
# (Import PipeLine library for image processing pipeline and performance timing)
from libs.PipeLine import PipeLine, ScopedTiming

# 设置图像处理分辨率常量
# (Set image processing resolution constants)
PICTURE_WIDTH = 160
PICTURE_HEIGHT = 120

# 初始化摄像头变量为空
# (Initialize camera variable as None)
sensor = None

# 设置显示分辨率常量
```

```python
# (Set display resolution constants)
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480


def scale_coordinates(data_tuple, target_resolution="640x480"):
    # 声明全局变量
    # (Declare global variables)
    global PICTURE_WIDTH, PICTURE_HEIGHT
    """
    将160x120分辨率下的坐标元组等比例缩放到目标分辨率
    (Scale coordinate tuple from 160x120 resolution proportionally to target
resolution)

    参数 (Parameters):
        data_tuple: 包含坐标信息的元组 (x1, y1, x2, y2)
                    (Tuple containing coordinate information (x1, y1, x2, y2))
        target_resolution: 目标分辨率，可选 "640x480" 或 "640x480"
                           (Target resolution, optional "640x480" or "640x480")

    返回 (Returns):
        包含缩放后坐标的新元组 (x1, y1, x2, y2, length)
        (New tuple containing scaled coordinates (x1, y1, x2, y2, length))
    """
    # 检查输入类型，确保是至少包含4个元素的元组
    # (Check input type, ensure it's a tuple with at least 4 elements)
    if not isinstance(data_tuple, tuple) or len(data_tuple) < 4:
        raise TypeError(f"期望输入至少包含4个元素的元组，但收到了
{type(data_tuple).__name__}")
        # (Expected a tuple with at least 4 elements, but received {type})

    # 从元组中解析坐标点
    # (Extract coordinates from the tuple)
    x1, y1, x2, y2 = data_tuple[:4]

    # 设置原始分辨率
    # (Set source resolution)
    src_width, src_height = PICTURE_WIDTH, PICTURE_HEIGHT

    # 根据目标分辨率参数设置目标宽高
    # (Set target width and height based on target resolution parameter)
    if target_resolution == "640x480":
        dst_width, dst_height = 640, 480
    elif target_resolution == "640x480":  # 注意：这里条件与上面相同，可能是代码错误
        # (Note: this condition is the same as above, might be a code error)
        dst_width, dst_height = 640, 480
    else:
        raise ValueError("不支持的分辨率，请使用 '640x480' 或 '640x480'")
        # (Unsupported resolution, please use '640x480' or '640x480')

    # 计算横向和纵向的缩放比例
    # (Calculate horizontal and vertical scaling ratios)
    scale_x = dst_width / src_width
    scale_y = dst_height / src_height

    # 对坐标进行缩放，并四舍五入保证是整数
    # (Scale coordinates and round to ensure integers)
    scaled_x1 = round(x1 * scale_x)
```

```
    scaled_y1 = round(y1 * scale_y)
    scaled_x2 = round(x2 * scale_x)
    scaled_y2 = round(y2 * scale_y)

    # 计算缩放后线段的长度（欧几里得距离）
    # (Calculate length of scaled line segment (Euclidean distance))
    dx = scaled_x2 - scaled_x1
    dy = scaled_y2 - scaled_y1
    length = round((dx**2 + dy**2)**0.5)

    # 返回缩放后的坐标元组
    # (Return tuple of scaled coordinates)
    return (scaled_x1, scaled_y1, scaled_x2, scaled_y2)

# 设置显示模式为LCD
# (Set display mode to LCD)

display_mode = "LCD"

# 创建图像处理Pipeline，设置RGB888格式尺寸和显示尺寸
# (Create image processing pipeline with RGB888 format size and display size)
pl = PipeLine(rgb888p_size=[640,360], display_size=[640,480],
display_mode=display_mode)
# 创建Pipeline实例，设置通道1的帧大小
# (Create pipeline instance, set frame size for channel 1)
pl.create(ch1_frame_size=[PICTURE_WIDTH,PICTURE_HEIGHT])

# 主循环
# (Main loop)
while True:
    # 从通道1捕获图像
    # (Capture image from channel 1)
    img = pl.sensor.snapshot(chn=CAM_CHN_ID_1)

    # 在图像中查找线段，合并距离为20，最大theta差异为5度
    # (Find line segments in the image, merge distance 20, max theta difference 5
degrees)
    lines = img.find_line_segments(merge_distance=15, max_theta_diff=10)

    # 创建一个新的ARGB8888格式的图像用于显示
    # (Create a new ARGB8888 format image for display)
    img = image.Image(640, 480, image.ARGB8888)

    # 清空图像
    # (Clear the image)
    img.clear()

    # 遍历找到的所有线段
    # (Iterate through all found line segments)
    for i, line in enumerate(lines):
        # 获取线段坐标并缩放到显示分辨率
        # (Get line segment coordinates and scale to display resolution)
        line = scale_coordinates(line.line())

        # 在图像上绘制红色线段，线宽为6
        # (Draw red line on the image with thickness 6)
        img.draw_line(line, color=(255,0,0), thickness=6)
```

```
    # 在OSD3层显示图像
    # (Display the image on OSD3 layer)
    Display.show_image(img, 0, 0, Display.LAYER_OSD3)

    # 短暂休眠微秒级延时，避免CPU过度占用
    # (Brief microsecond sleep to avoid excessive CPU usage)
    time.sleep_us(1)
```

代码讲解

本节代码的基本结构如下

导入和初始化部分：

- 导入了基础模块（time, os, sys, gc）用于系统操作和内存管理
- 导入了媒体相关模块，用于处理图像采集、显示等功能
- 导入了PipeLine库，用于图像处理Pipeline和性能计时
- 定义了两组分辨率常量：
    - 图像处理分辨率：160x120
    - 显示分辨率：640x480

`scale_coordinates`函数：

- 功能：将160x120分辨率下的坐标等比例转换到640x480分辨率
- 输入：包含坐标的元组(x1, y1, x2, y2)
- 处理步骤：
    - 输入验证
    - 计算缩放比例
    - 对坐标进行等比例转换
    - 计算线段长度
- 返回：转换后的坐标元组

图像处理Pipeline设置：

- 创建PipeLine实例，配置：
    - RGB888格式尺寸：640x360
    - 显示尺寸：640x480
    - 显示模式：LCD
- 设置通道1的帧大小为160x120

主循环处理：

- 循环执行以下步骤：
    - 从摄像头捕获图像
    - 在图像中查找线段（合并距离15，最大角度差10度）
    - 创建新的显示图像（640x480，ARGB8888格式）
    - 对每个检测到的线段：
        - 将坐标转换到显示分辨率
        - 用红色绘制线段（粗细为6）
    - 在显示层OSD3显示处理后的图像
    - 短暂休眠以控制CPU使用率

> 这一节代码中，我们没有使用更常规的方式去做线段检测，而是借助K230支持多个图层的特性
>
> 用较低的分辨率去检测图形，然后放缩结果到高分辨的背景图像中
>
> 这样操作后，程序运行的帧率会显著的增高
>
> 本章后续的所有代码则用的是原始的检测方法，代码会更简单，但是帧率会降低不少

检测线段

find_line_segments

```
image.find_line_segments([roi[, merge_distance=0[, max_theta_difference=15]]])
```

使用**霍夫转换**来查找图像中的线段。返回一个 image.line 对象的列表。

roi 是一个用以复制的矩形的感兴趣区域(x, y, w, h)。如果未指定， ROI 即图像矩形。操作范围仅限于roi区域内的像素。

merge_distance 指定两条线段之间的可以相互分开而不被合并的最大像素数。

max_theta_difference 是上面 merge_distancede 要合并的的两个线段的最大角度差值。

> 此方法使用LSD库（也被OpenCV使用）来查找图像中的线段。这有点慢，但是非常准确，线段不会跳跃。
>
> 不支持压缩图像和bayer图像。

霍夫变换

霍夫变换是一种从图像中检测直线、圆等几何形状的数学方法。

> 打个比方，想象你面前有一张散落着很多点的图纸。
>
> 如果要找出这些点能组成的直线:
>
> 1. 传统方法是尝试连接任意两点看是否能形成直线,这样很费时
> 2. 霍夫变换的思路是反过来 - 对每个点,假设它可能在无数条直线上,找到多个点共同落在的那条直线
>
> 举个生活中的例子:你站在路边看电线杆,虽然电线杆分布在不同位置,但是你一眼就能看出它们排成了一条直线。这就类似霍夫变换的原理。

实际应用

- 车道线检测：自动驾驶中识别道路标线
- 文档扫描：找出文档边界
- 建筑识别：检测建筑物的边缘线条

算法步骤

- 首先对图像进行边缘检测
- 建立参数空间的累加器数组
- 对每个边缘点进行参数空间投票
- 寻找累加器中的局部最大值
- 根据这些最大值反推出原图中的直线